

ЗАВДАННЯ 5: РЕАЛІЗАЦІЯ БАГАТОШАРОВОГО ПЕРЦЕПТРОНУ З НУЛЯ В PYTORCH

Це завдання направлено на глибоке розуміння фундаментальних принципів роботи нейронних мереж. Замість використання високорівневих інструментів PyTorch, ви реалізуєте всі компоненти самостійно - від обчислення градієнтів до оновлення ваг. Це дозволить зрозуміти, що відбувається "під капотом" сучасних бібліотек машинного навчання.

Dataset: [Link](#). Wine Quality Dataset містить фізико-хімічні властивості вин. Завдання - передбачити якість вина за цими властивостями. Це типова задача багатокласової класифікації, де нам потрібно розрізнити 6 різних класів якості.

1. Підготовка даних

Пояснення: Правильна підготовка даних - ключ до успішного навчання нейронної мережі. Ми нормалізуємо ознаки, щоб усі вони знаходились в одному масштабі, що пришвидшує збіжність. Розділення на три вибірки дозволяє контролювати перенавчання та об'єктивно оцінювати якість моделі.

```
def load_and_preprocess_data():  
    """  
    Кроки:  
    1. Завантажити CSV з даними про вино  
    2. Відокремити ознаки (X) та мітки (y)  
    3. Нормалізувати ознаки за допомогою StandardScaler  
    4. Розділити дані на тренувальну/валідаційну/тестову вибірки  
    5. Конвертувати в тензори PyTorch
```

2. Функції активації (реалізація з нуля)

Пояснення: Функції активації вводять нелінійність у модель, що дозволяє мережі навчатися складні шаблони. Кожна функція має реалізацію як для forward pass, так і для backward pass (похідна), оскільки нам потрібно знати, як змінювати ваги під час навчання.

```
class ActivationFunctions:

    @staticmethod

    def relu(x, derivative=False):

        # ReLU:  $f(x) = \max(0, x)$ 

        # Похідна: 1 для  $x > 0$ , 0 для  $x \leq 0$ 

        if derivative:

            return (x > 0).float() # Похідна для backpropagation

    @staticmethod

    def tanh(x, derivative=False):

        # Tanh:  $f(x) = (e^x - e^{-x}) / (e^x + e^{-x})$ 

        # Похідна:  $1 - \tanh^2(x)$ 

        if derivative:

            return 1 - torch.tanh(x)**2 # Похідна через тотожність
```

3. Функція втрат (реалізація з нуля)

Пояснення: Categorical Cross-Entropy вимірює, наскільки прогнози моделі відрізняються від справжніх міток. Особливість нашої реалізації - ми оптимізуємо обчислення похідної для комбінації softmax + cross-entropy, що є стандартним підходом у нейронних мережах.

```
class LossFunctions:

    @staticmethod

    def categorical_cross_entropy(y_pred, y_true, derivative=False):
```

```

if derivative:

    # Оптимізована похідна для softmax + cross_entropy

    # Градієнт = (y_pred - y_true) для one-hot представлення

    grad = y_pred.clone()

    grad[range(batch_size), y_true] -= 1

    return grad / batch_size

```

4. Власна реалізація MLP

Пояснення: Це ядро нашого завдання. Ми створюємо повноцінну нейронну мережу, яка може навчатися без використання autograd. Ключові аспекти: кешування проміжних значень під час forward pass для ефективного backward pass та правильне застосування ланцюгового правила для обчислення градієнтів.

```

class CustomMLP:

    def __init__(self, layer_sizes, activations, learning_rate=0.01):

        """

        Ініціалізація включає:

        - Збереження архітектури мережі

        - Ініціалізацію ваг за методом Xavier

        - Налаштування швидкості навчання

        """

    def forward(self, X, training=True):

        """

        Прохід даних через мережу:

        Для кожного шару:  $Z = W \cdot A_{prev} + b$ ,  $A = \text{activation}(Z)$ 

```

```

Кешуємо Z та A для кожного шару для backprop
"""

cache = {'A0': X.T}

for i in range(1, len(self.layer_sizes)):

    Z = self.parameters[f'W{i}'] @ A_prev + self.parameters[f'b{i}']

    A = activation_func(Z)

    cache[f'Z{i}'] = Z # Зберігаємо для backprop

    cache[f'A{i}'] = A


def backward(self, X, y, cache):
    """
    Зворотній прохід:

    1. Обчислюємо градієнт для вихідного шару
    2. Поширюємо градієнти назад через шари
    3. Використовуємо ланцюгове правило
    4. Оновлюємо ваги та зміщення
    """

    # Градієнти для вихідного шару

    dZ_output = dLoss/dOutput * dOutput/dZ_output


    # Поширення назад через приховані шари

    for layer in reversed(hidden_layers):

        dA = dZ_next @ W_next.T

        dZ = dA * activation_derivative(Z_current)

```

5. PyTorch реалізація для порівняння

Пояснення: Ця реалізація слугує еталоном для порівняння. Ми використовуємо вбудовані механізми PyTorch, щоб переконатися, що наша власна реалізація працює коректно. Важливо використовувати однакові гіперпараметри для обох реалізацій.

```
class PyTorchMLP(nn.Module):

    def __init__(self, input_size, hidden_sizes, output_size):

        # Використовуємо nn.Sequential для створення мережі

        # Аналогічна архітектура до нашої власної реалізації


def train_pytorch_model(model, X_train, y_train, X_val, y_val, epochs=1000):

    """

    Навчання з використанням вбудованих інструментів PyTorch:

    - Automatic differentiation (autograd)

    - Оптимізатори

    - Вбудовані функції втрат

    """

    criterion = nn.CrossEntropyLoss() # Включає softmax + cross-entropy

    optimizer = torch.optim.Adam(model.parameters())
```

6. Візуалізація та аналіз

Пояснення: Візуалізація допомагає зрозуміти процес навчання та виявити потенційні проблеми. Ми порівнюємо не лише фінальну точність, але й динаміку навчання, стабільність, та розподіл помилок за класами.

```
def plot_training_history(history_custom, history_pytorch):
```

```
    """
```

```
    Аналізуємо:
```

- Швидкість збіжності
- Наявність overfitting (велика різниця між train та val loss)
- Стабільність навчання

```
    """
```

```
def plot_confusion_matrix(y_true, y_pred_custom, y_pred_pytorch, class_names):
```

```
    """
```

```
    Показуємо:
```

- Які класи плутаються найбільше
- Чи однакові патерни помилок в обох моделях
- Загальну якість класифікації

Очікується, що власна реалізація буде працювати дещо повільніше, але матиме подібну якість до PyTorch версії. Невелика різниця в точності (до 5%) є прийнятною і може бути пов'язана з різними методами ініціалізації або оптимізації.

ПИТАННЯ:

1. Похідна комбінації Softmax + Cross-Entropy

Доведіть, що для комбінації Softmax активації та Cross-Entropy втрат, градієнт по відношенню до логітів вихідного шару обчислюється як:

$$\partial L / \partial z_i = y_{\text{pred}_i} - y_{\text{true}_i}$$

де y_{true} - one-hot вектор справжніх міток. Поясніть, чому ця формула така проста та які математичні властивості роблять можливим таке спрощення.

2. Аналіз зміни градієнтів у глибоких мережах

Розглянемо мережу з 5 шарами, де кожен шар використовує ReLU активацію. Похідна ReLU дорівнює 0 для від'ємних значень та 1 для додатних.

- Як буде змінюватися градієнт для першого шару після 100 ітерацій навчання, якщо 60% нейронів у прихованих шарах мають від'ємні значення?
- Обчисліть максимально можливе значення градієнту для ваг першого шару після N шарів з ReLU.

3. Умовність матричних операцій у backpropagation

У нашій реалізації backpropagation ми використовуємо транспонування матриць:

$$dW = (1/m) * dZ @ A_{\text{prev}.T}$$

Поясніть, чому саме така форма матричного множення є правильною з точки зору розмірностей.

- Виведіть загальну формулу для градієнта матриці W у випадку, коли $Z = WX + b$, де X - матриця вхідних даних.

4*. Аналіз власних значень гессіана

Для простої двошарової мережі з одним прихованим шаром:

- Запишіть матрицю Гессіана для функції втрат відносно ваг першого шару.
- Як власні значення цієї матриці пов'язані зі швидкістю збіжності градієнтного спуску?
- Поясніть, чому в глибоких мережах часто виникає проблема "зникаючих" градієнтів з точки зору власних значень.